Laporan UTS

Arsitektur & Perancang Perangkat Lunak

Disusun Oleh :

41425075_Nova Evelyn Sirait

Sarjana Terapan Teknologi rekayasa Perangkat Lunak

IT Del Angkatan 2025

Fakultas Vokasi

CASE STUDY 1

Perancangan Arsitektur Sistem MediTrack

1. **Design Thinking & Architecture Selection**

   Dalam merancang sistem MediTrack, arsitektur yang dipilih adalah **Hybrid Architecture**, yaitu kombinasi antara **Microservices Architecture** dan pendekatan **Domain-Driven Design (DDD)**.

   Menurut saya, pendekatan ini paling cocok karena sistem MediTrack bukan sistem sederhana. Ada banyak fitur seperti appointment, rekam medis, pembayaran, dan pemesanan obat yang masing-masing punya kompleksitas sendiri.

   a. **Justifikasi Pemilihan Arsitektur**

      Beberapa prinsip arsitektur yang menjadi dasar pemilihan:

      1. **Separation of Concerns**

         Setiap service memiliki tanggung jawab yang jelas. Misalnya, Appointment Service hanya mengurus jadwal dan booking, tidak bercampur dengan pembayaran atau data user.

      2. **Loose Coupling dan High Cohesion**

         Antar service tidak saling bergantung secara langsung (loose coupling), tetapi setiap service fokus pada satu fungsi utama (high cohesion). Ini membuat sistem lebih fleksibel saat ada perubahan.

      3. **Scalability**

         Dengan microservices, bagian yang sering digunakan seperti Appointment Service bisa di-scale tanpa harus mempengaruhi service lain.

   b. **Trade-offs dan Keterbatasan**

      Meskipun memiliki banyak kelebihan, ada beberapa kekurangan:

      - Sistem menjadi lebih kompleks dibanding monolith
      - Membutuhkan infrastruktur tambahan (API Gateway, Message Broker)
      - Monitoring dan debugging lebih sulit
      - Komunikasi antar service menambah latency

   c. **Dukungan terhadap Sistem**

      Scalability:

      Setiap service bisa dikembangkan dan di-scale secara terpisah.

Maintainability:

Jika ada perubahan pada satu fitur, tidak perlu mengubah seluruh sistem.

Extensibility:

Mudah menambahkan fitur baru, misalnya menambah service untuk telemedicine.

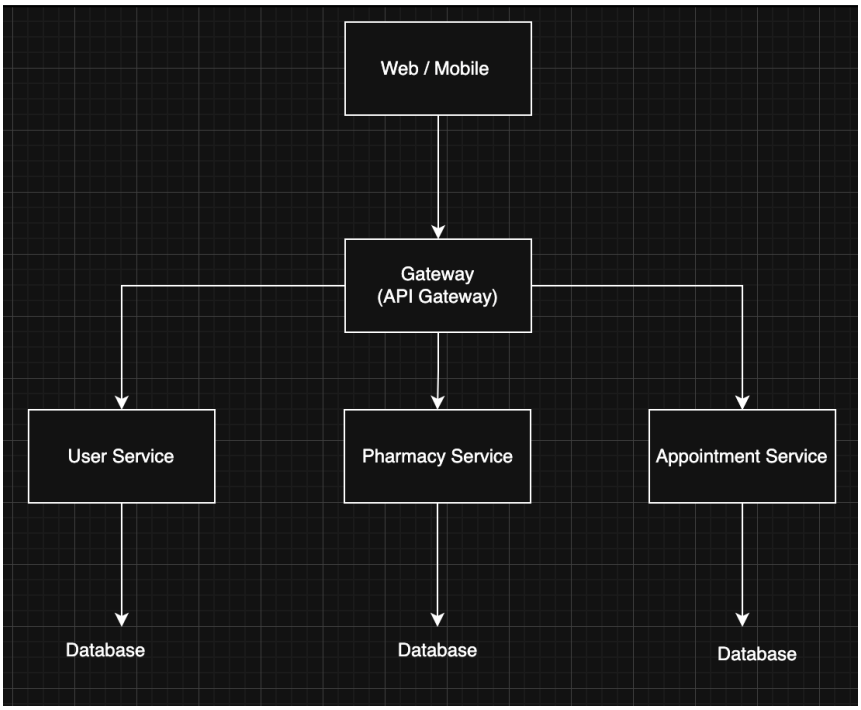
2. **System Decomposition & Modeling**
   a. **Daftar Module / Services**
      - Sistem MediTrack dibagi menjadi beberapa service utama:
      - **User Service** → Mengelola data user (login, registrasi)
      - **Appointment Service** → Mengatur jadwal dan booking dokter
      - **EHR Service** → Menyimpan rekam medis pasien
      - **Pharmacy Service** → Mengelola pemesanan obat
      - **Payment Service** → Mengelola transaksi pembayaran
      - **Notification Service** → Mengirim notifikasi

   b. **Key Classes / Entities**

      Beberapa entitas utama dalam sistem:
      - **User** → menyimpan data pengguna
      - **Appointment** → menyimpan jadwal dan booking
      - **Doctor** → data dokter
      - **MedicalRecord** → rekam medis pasien
      - **Order** → pemesanan obat
      - **Payment** → transaksi pembayaran

      Relasi antar entitas:
      - User → Appointment (1:N)
      - Appointment → Doctor (N:1)
      - User → MedicalRecord (1:N)
      - Order → Medicine (N:M)
      - Payment → Order (1:1)

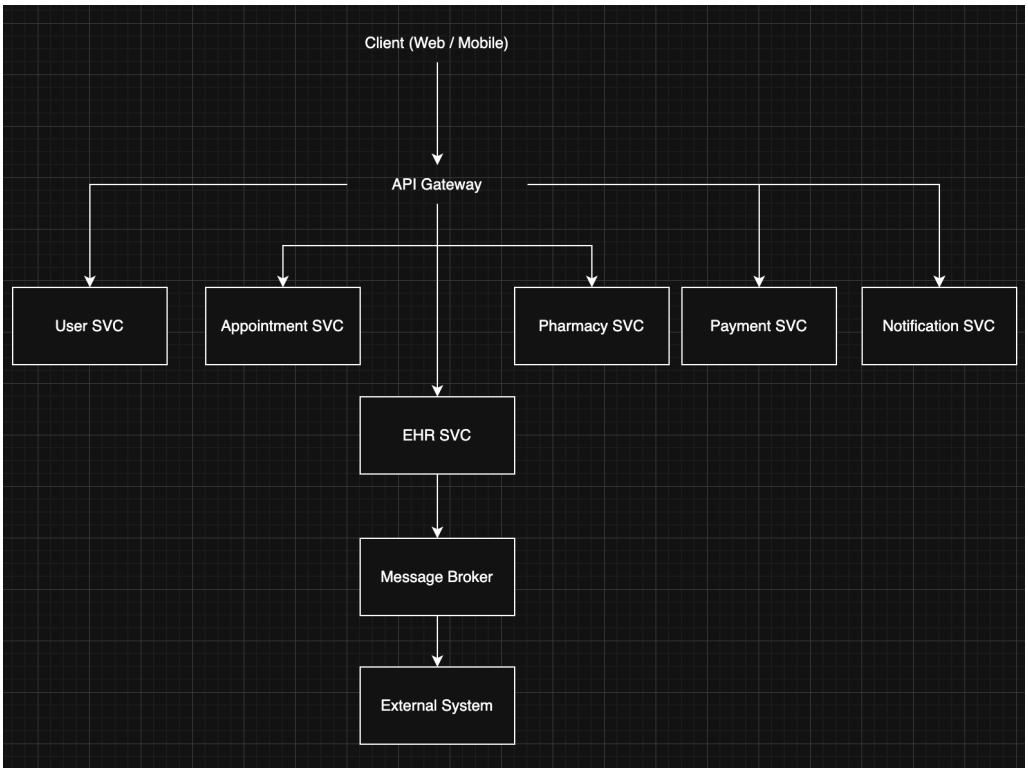
c. Component Diagram



Penjelasan:

Diagram ini menunjukkan bahwa sistem dibagi menjadi beberapa service yang dihubungkan melalui API Gateway. Setiap service memiliki database masing-masing.

3. Architecture Visualization



a. **Komponen dan Data Flow**
    - User mengakses sistem melalui client
    - Request masuk ke API Gateway
    - API Gateway meneruskan ke service terkait
    - Service menyimpan data ke database
    - Event dikirim melalui message broker
b. **Boundary Sistem**

    Service Boundary:

    Setiap service berdiri sendiri (User, Appointment, dll)

    Bounded Context (DDD):
    - Booking Context → Appointment
    - Healthcare Context → HER
    - Commerce Context → Payment & Pharmacy
c. **Integrasi dengan External System**
    - Payment Gateway → Payment Service
    - Pharmacy API → Pharmacy Service
    - Sistem Lab → EHR Service

KESIMPULAN

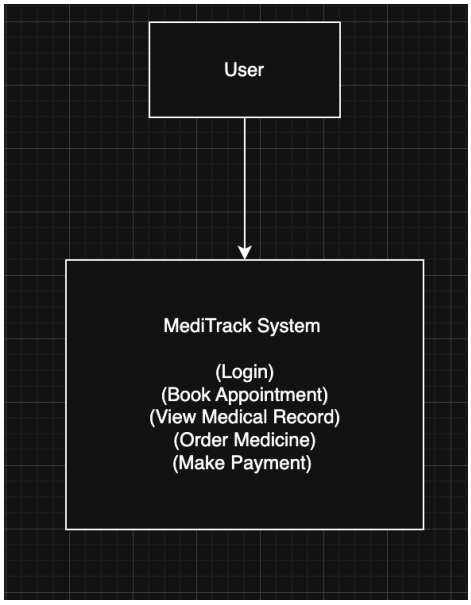
Berdasarkan perancangan yang telah dilakukan, penggunaan arsitektur hybrid (microservices + DDD) memberikan banyak keuntungan dalam pengembangan sistem MediTrack. Sistem menjadi lebih scalable, fleksibel, dan mudah dikembangkan. Namun, kompleksitas sistem juga meningkat sehingga diperlukan pengelolaan yang baik.

CASE STUDY 2

Migrasi Monolithic ke Microservices pada Sistem MediTrack

1. **Use Case**

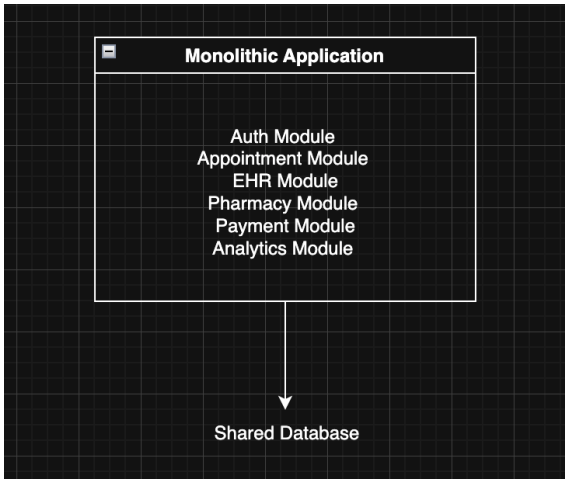
    Untuk memahami sistem secara sederhana, use case digunakan untuk menggambarkan interaksi antara user dengan sistem MediTrack.



Penjelasan:

Pada sistem ini, user dapat melakukan beberapa aktivitas utama seperti login, melakukan booking appointment dengan dokter, melihat rekam medis, memesan obat, dan melakukan pembayaran. Use case ini tidak berubah meskipun arsitektur sistem mengalami perubahan, karena yang berubah adalah cara sistem dibangun, bukan fitur yang disediakan.

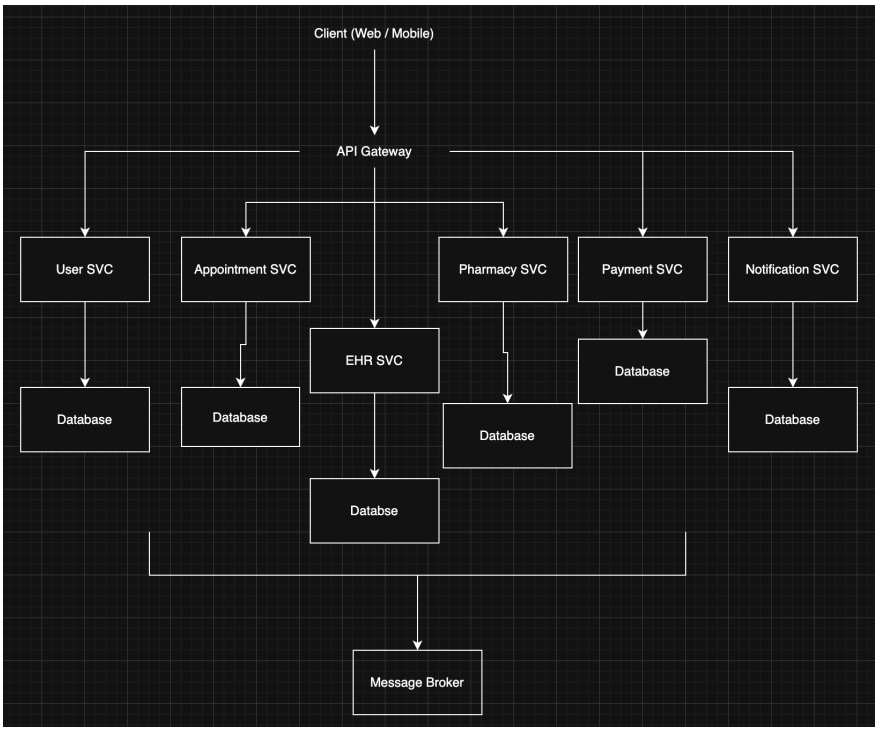
2. **Architecture Diagram**
   1. **Arsitektur Monolith (Sebelum Migrasi)**



Penjelasan:

Pada arsitektur monolith, semua modul berada dalam satu aplikasi besar dan menggunakan satu database yang sama. Hal ini membuat sistem menjadi sulit dikembangkan karena setiap perubahan kecil dapat mempengaruhi keseluruhan sistem.

2. **Arsitektur Microservices (Setelah Migrasi)**



3. **Coverage Sistem Lama ke Baru**

   Pada arsitektur baru, seluruh fungsi dari sistem monolith tetap dipertahankan, namun dipisahkan menjadi service terpisah:

   - Auth Module → Auth Service
   - Appointment → Appointment Service
   - EHR → EHR Service
   - Pharmacy → Pharmacy Service
   - Payment → Payment Service
   - Analytics → Analytics Service

   Penjelasan:

   Dengan pendekatan ini, sistem lama tidak langsung dihapus, tetapi secara bertahap digantikan oleh service baru.

3. **Reasoning & Trade-offs**

   Alasan Transformasi

   Perubahan dari monolith ke microservices dilakukan karena beberapa kendala pada sistem lama, seperti sulitnya melakukan scaling dan lamanya proses deployment. Dengan microservices, setiap bagian sistem dapat dikembangkan dan dijalankan secara independen.

   Trade-offs

Meskipun microservices memiliki banyak kelebihan, terdapat beberapa konsekuensi yang harus diperhatikan:

- Sistem menjadi lebih kompleks
- Dibutuhkan infrastruktur tambahan seperti API Gateway dan Message Broker
- Proses debugging lebih sulit
- Komunikasi antar service menambah latency

Namun, kelebihan seperti scalability dan fleksibilitas tetap menjadi alasan utama penggunaan arsitektur ini.

4. **Migration Strategy**

   Strategi migrasi yang digunakan adalah **Strangler Fig Pattern**.

   Tahapan migrasi:

   1. Menambahkan API Gateway sebagai pintu masuk utama
   2. Memisahkan satu modul terlebih dahulu (misalnya Auth)
   3. Mengarahkan request ke service baru
   4. Membiarkan sistem monolith tetap berjalan
   5. Mengulangi proses hingga semua modul terpisah

   Penjelasan:

   Pendekatan ini memungkinkan migrasi dilakukan secara bertahap tanpa mengganggu sistem yang sedang berjalan.

5. **Implementasi – Monolith Representation (Java)**

   Berikut adalah contoh sederhana struktur monolith menggunakan Java:

   Penjelasan:

   Pada contoh ini, semua modul berada dalam satu aplikasi dan saling terhubung langsung. Hal ini menunjukkan bahwa sistem bersifat **tightly coupled**.

6. **Implementasi – Microservices Mapping**

   Mapping Modul ke Service

Modul Monolith	Microservice
Auth	Auth Service
Appointment	Appointment Service

EHR	EHR Service
Pharmacy	Pharmacy Service
Payment	Payment Service
Analytics	Analytics Service

Perubahan Komunikasi

Sebelum (Monolith):

- Komunikasi langsung antar class
- Menggunakan satu database

Sesudah (Microservices):

- Komunikasi melalui REST API
- Menggunakan message broker
- Database terpisah per service

Contoh Komunikasi REST API

Contoh Event (Message Queue)

KESIMPULAN

Berdasarkan hasil analisis, migrasi dari monolithic ke microservices memberikan banyak keuntungan seperti peningkatan scalability, fleksibilitas, dan kemudahan pengembangan. Namun, pendekatan ini juga meningkatkan kompleksitas sistem sehingga diperlukan perencanaan dan strategi migrasi yang matang.